Software Verification

Master Programme in Computer Science

Mário Pereira mjp.pereira@fct.unl.pt

Nova School of Science and Technology, Portugal

October 12, 2026

Lecture 5

based on previous editions by João Seco, Luís Caires, and Bernardo Toninho

based on lectures by Andrei Paskevich, Jean-Christophe Filliâtre and Claude Marché

Verification of Imperative Programs (2/n)

1. Hoare Logic Recap
2. Hoare Logic Rule for Loops
3. Loop Invariants
4. Weakest Pre-condition Calculus

Floyd-Hoare logic – Recap

Initially (1970): axiomatic semantics of programs

Inference rules to construct valid triples:

$$\frac{}{\{P\}\,\texttt{skip}\,\{P\}}$$

$$\frac{}{\{P[x \mapsto t]\}\,x := t\,\{P\}}$$

$$\frac{\{P\}\,e_1\,\{Q\} \qquad \{Q\}\,e_2\,\{R\}}{\{P\}\,e_1\ ;\ e_2\,\{R\}}$$

Notation $P[x \mapsto t]$: replace in P every occurrence of x with t

Consequence rule:

$$\frac{\models P \implies P' \qquad \{P'\}\, e\, \{Q'\} \qquad \models Q' \implies Q}{\{P\}\, e\, \{Q\}}$$

Example: proof of $\{x = 1\}\, x := x + 2\, \{x = 3\}$

$$\dfrac{\models x = 1 \to x + 2 = 3 \qquad \dfrac{\begin{array}{c}(x = 3)[x \mapsto x + 2] \equiv x + 2 = 3 \\ \vdots \\ \{x + 2 = 3\}\, x := x + 2\, \{x = 3\}\end{array}}{\{x = 1\}\, x := x + 2\, \{x = 3\}}$$

Rules of Hoare Logic (general form) – Recap

The inference rules of Hoare logic are used to derive (valid) Hoare triples given some already derived Hoare triples.

$$\frac{\{A_1\}\, P_1\, \{B_1\} \qquad \ldots \qquad \{A_n\}\, P_n\, \{B_n\}}{\{A\}\, C(P_1, \ldots, P_n)\, \{B\}}$$

What is nice here:

- the program in the conclusion contains the subprograms $P_1, \ldots, P_n$ as components
- we derive properties of the composite from the properties of its parts (compositionality)
- pretty much the same as with a type system

Function definition – Recap

We may want to delegate some functionality to a function:

$\texttt{let}\ f\ (v_1 : \tau_1)\ \ldots\ (v_n : \tau_n) : \tau\ C = e$	defined function
$\texttt{val}\ f\ (v_1 : \tau_1)\ \ldots\ (v_n : \tau_n) : \tau\ C$	abstract function

Function behavior is specified with a contract:

C	$::=$	$\texttt{requires}\ P$	precondition
		$\texttt{writes}\ x_1 \ldots\ x_n$	modified global references
		$\texttt{ensures}\ Q$	postcondition

Function definition – Recap

Declarations annotated with pre- and post- conditions.

A program P is a set of function declarations.

Each function declaration is validated as follows:

1. assume its pre-condition
2. prove its post-condition.

$$\frac{\{P\ v_1\ \ldots\ v_n\}\, e\, \{Q\ v_1\ \ldots\ v_n\ \mathscr{R})\}}{\begin{array}{l}\texttt{let}\ f\ (v_1\ :\ \tau_1)\ \ldots\ (v_n\ :\ \tau_n)\ :\ \tau \\ \texttt{requires}\ P\ v_1\ \ldots\ v_n \\ \texttt{ensures}\ Q\ v_1\ \ldots\ v_n\ \mathscr{R}\ =\ e\end{array}}$$

Function calls – Recap

$$\texttt{let}\ f\ (v_1\ :\ \tau_1)\ \ldots\ (v_n\ :\ \tau_n)\ :\ \tau$$
$$\texttt{requires}\ P\ v_1\ \ldots\ v_n$$
$$\texttt{ensures}\ Q\ v_1\ \ldots\ v_n\ \mathscr{R}\ =\ e$$

$$\frac{A \Rightarrow P\ e_1\ \ldots\ e_n \qquad Q\ e_1\ \ldots\ e_n\ \mathscr{R} \Rightarrow B[x \mapsto r]}{\{A\}\, x := f\ e_1\ \ldots\ e_n\, \{B\}}$$

Each function call is validated as follows:

1. prove the (instantiated) pre-condition of f

$$A \implies P\ e_1\ \ldots\ e_n$$

2. assume the (instantiated) post-condition of f

$$Q\ e_1\ \ldots\ e_n\ \mathscr{R} \implies B[x \mapsto \mathscr{R}]$$

Function calls – Recap

$$\texttt{let}\ f\ (v_1\ :\ \tau_1)\ \ldots\ (v_n\ :\ \tau_n)\ :\ \tau$$
$$\texttt{requires}\ P\ v_1\ \ldots\ v_n$$
$$\texttt{ensures}\ Q\ v_1\ \ldots\ v_n\ \mathscr{R}\ =\ e$$

$$\frac{A \Rightarrow P\ e_1\ \ldots\ e_n \qquad Q\ e_1\ \ldots\ e_n\ \mathscr{R} \Rightarrow B[x \mapsto r]}{\{A\}\, x := f\ e_1\ \ldots\ e_n\, \{B\}}$$

Calls are opaque. We only know what is in the post-condition.

Verification with function calls is modular.

Rules for `if` and `while`:

$$\frac{\{P \wedge t\}\, e_1\, \{Q\} \qquad \{P \wedge \neg t\}\, e_2\, \{Q\}}{\{P\}\, \texttt{if}\ t\ \texttt{then}\ e_1\ \texttt{else}\ e_2\, \{Q\}}$$

$$\frac{\{J \wedge t\}\, e\, \{J\}}{\{J\}\, \texttt{while}\ t\ \texttt{do}\ e\, \{J \wedge \neg t\}}$$

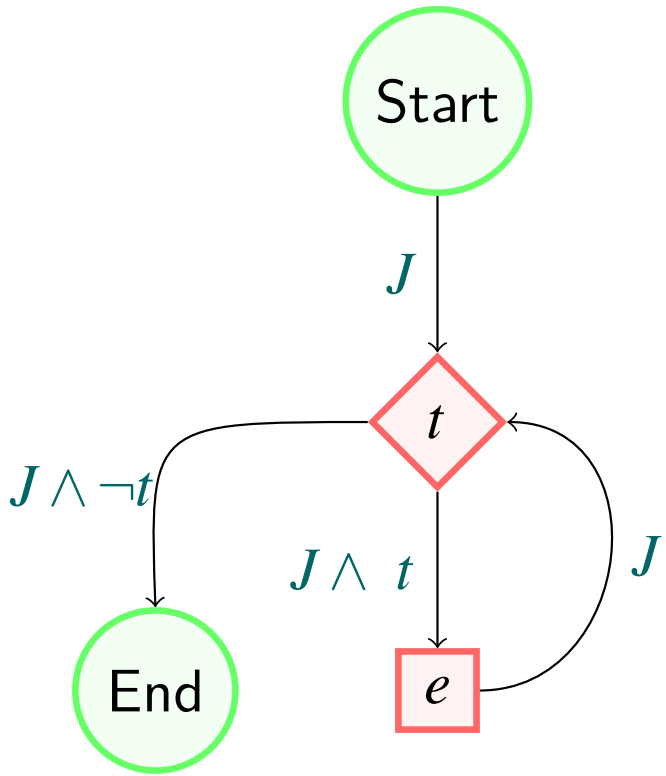
Formula J is a loop invariant.

Finding a right invariant is a major difficulty.

Loops in Hoare Logic

Hoare Logic Rule for Loops – In a Picture

$$\frac{\{J \wedge t\}\, e\, \{J\}}{\{J\}\, \texttt{while}\ t\ \texttt{do}\ e\ \{J \wedge \neg t\}}$$



Hoare Logic Rule for Loops – In Words

$$\frac{\{J \wedge t\}\, e\, \{J\}}{\{J\}\ \texttt{while}\ t\ \texttt{do}\ e\ \{J \wedge \neg t\}}$$

J	Invariant initially valid
$J \wedge t$	Loop condition is true
e	Execution of the loop body
J	Invariant re-established
$J \wedge t$	Loop condition is true
e	Execution of the loop body
J	Invariant re-established
$\vdots$	(any number of iterations)
J	Invariant re-established
$J \wedge \neg t$	Loop exits; condition is false

Hoare Logic Rule for Loops

J = Invariant Condition

$$\frac{\{J \wedge t\}\, e\, \{J\}}{\{J\}\, \texttt{while}\ t\ \texttt{do}\ e\ \{J \wedge \neg t\}}$$

We cannot predicate in general for how many iterations the `while` loop will run (undecidability of the halting problem).

We approximate all iterations by an invariant condition.

A loop invariant holds at loop entry and at loop exit.

Hoare Logic Rule for Loops

J = Invariant Condition

$$\frac{\{J \wedge t\}\, e\, \{J\}}{\{J\}\, \texttt{while}\ t\ \texttt{do}\ e\ \{J \wedge \neg t\}}$$

If the invariant holds initially and is preserved by the loop body, it will hold when the loop terminates.

No matter how many iterations will run.

Finding the good invariant requires patience and creativity.

Hoare Logic Rule for Loops

J = Invariant Condition

$$\frac{\{J \wedge t\}\, e\, \{J\}}{\{J\}\, \texttt{while}\ t\ \texttt{do}\ e\ \{J \wedge \neg t\}}$$

The invariant describes the state in any possible iteration.

The invariant works like the induction hypothesis in a proof.

- The base case is the loop executed 0 times,
- the loop body is the induction step that iterates from n to $n+1$,
- There must exist a valid induction measure.

Loop Invariants – Example

```
i := 0;
while i < n do {
  i := i + 1;
}
```

Loop Invariants – Example

$\{0 \leq n\}$

```
i := 0;
while i < n do {
  i := i + 1;
}
```

$\{i == n\}$

Loop Invariants – Example

$\{0 \leq n\}$

```
i := 0;
```

$\{i == 0 \wedge 0 \leq n\}$

$\{0 \leq i \leq n\}$

```
while i < n do {
```

$\{0 \leq i \leq n \wedge i < n\}$

$\{0 \leq i < n\}$

$\{0 \leq i + 1 \leq n\}$

```
  i := i + 1;
```

$\{0 \leq i \leq n\}$

```
}
```

$\{0 \leq i \leq n \wedge i == n\}$

$\{i == n\}$

Loop Invariants, in Practice

Loop Invariants – Challenge

Consider the following program, lets call it P:

```
s := 0;
i := 0;
while i < n do {
  i := i + 1;
  s := s + i;
}
```

What is a good specification for P? What does P compute?

Loop Invariants – Challenge

Consider the following program, lets call it P:

```
s := 0;
i := 0;
while i < n do {
  i := i + 1;
  s := s + i;
}
```

What is a good specification for P? What does P compute?

$$\{0 \leq n\}\ \mathtt{P}\ \{s == \sum_{j=0}^{n} j\}$$

Is this a good specification for P?

Lets resort to Hoare Logic to mechanically check this triple.

Checking Loop Invariants

$\{0 \leq n\}$

```
s := 0;
i := 0;
while i < n do {
  i := i + 1;
  s := s + i;
}
```

$\{s == \sum_{j=0}^{n} j\}$

Checking Loop Invariants

$\{0 \leq n\}$

```
s := 0;
```

$\{s == 0 \wedge 0 \leq n\}$

```
i := 0;
```

$\{i == 0 \wedge s == 0 \wedge 0 \leq n\}$

```
while i < n do {
  i := i + 1;
  s := s + i;
}
```

$\{s == \sum_{j=0}^{n} j\}$

Checking Loop Invariants

$\{0 \leq n\}$

```
s := 0;
```

$\{s == 0 \wedge 0 \leq n\}$

```
i := 0;
```

$\{i == 0 \wedge s == 0 \wedge 0 \leq n\}$

$\{s == 0 \wedge 0 \leq i \leq n\}$

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$

```
while i < n do {
  i := i + 1;
  s := s + i;
}
```

$\{s == \sum_{j=0}^{n} j\}$

Checking Loop Invariants

$\{0 \leq n\}$

```
s := 0;
```

$\{s == 0 \wedge 0 \leq n\}$

```
i := 0;
```

$\{i == 0 \wedge s == 0 \wedge 0 \leq n\}$

$\{s == 0 \wedge 0 \leq i \leq n\}$

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$

```
while i < n do {
  i := i + 1;
  s := s + i;
}
```

$\{i == n \wedge s == \sum_{j=0}^{i} j\}$

$\{s == \sum_{j=0}^{n} j\}$

Checking Loop Invariants

$\{0 \leq n\}$

```
s := 0;
```

$\{s == 0 \wedge 0 \leq n\}$

```
i := 0;
```

$\{i == 0 \wedge s == 0 \wedge 0 \leq n\}$

$\{s == 0 \wedge 0 \leq i \leq n\}$

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$ *//invariant initially holds*

```
while i < n do {
```

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n \wedge i < n\}$ *//invariant holds at the beginning of iteration*

```
  i := i + 1;
  s := s + i;
```

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$ *//invariant is maintained*

```
}
```

$\{i == n \wedge s == \sum_{j=0}^{i} j\}$

$\{s == \sum_{j=0}^{n} j\}$

Breaking Loop Invariants

The loop invariant:

- may be broken inside the loop body
- must be re-established at the end.

Notice how the assignment rule

$$\{P[x \mapsto t]\}\, x := t\, \{P\}$$

breaks the invariant...

$$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n \wedge i < n\}$$

```
i := i + 1;
```

$$\{s == \sum_{j=0}^{i-1} j \wedge 0 \leq i-1 \leq n \wedge i-1 < n\}$$

$$\{s == \sum_{j=0}^{i-1} j \wedge 0 \leq i \leq n\}$$

Breaking Loop Invariants

The loop invariant:

- may be broken inside the loop body
- must be re-established at the end.

Notice how the assignment rule

$$\{P[x \mapsto t]\}\, x := t\, \{P\}$$

and then re-establishes it

$$\{s == \sum_{j=0}^{i-1} j \wedge 0 \leq i \leq n\}$$

```
s := s + i;
```

$$\{s == (\sum_{j=0}^{i-1} j) + i \wedge 0 \leq i \leq n\}$$

$$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$$

Checking Loop Invariants – the big picture

$\{0 \leq n\}$

```
s := 0;
```

$\{s == 0 \wedge 0 \leq n\}$

```
i := 0;
```

$\{i == 0 \wedge s == 0 \wedge 0 \leq n\}$

$\{s == 0 \wedge 0 \leq i \leq n\}$

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$ *//invariant initially holds*

```
while i < n do {
```

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n \wedge i < n\}$ *//invariant holds at the beginning of iteration*

```
  i := i + 1;
  s := s + i;
```

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$ *//invariant is maintained*

```
}
```

$\{i == n \wedge s == \sum_{j=0}^{i} j\}$

$\{s == \sum_{j=0}^{n} j\}$

Checking Loop Invariants – the big picture

$\{0 \leq n\}$

```
s := 0;
```

$\{s == 0 \wedge 0 \leq n\}$

```
i := 0;
```

$\{i == 0 \wedge s == 0 \wedge 0 \leq n\}$

$\{s == 0 \wedge 0 \leq i \leq n\}$

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$ *//invariant initially holds*

```
while i < n do {
```

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n \wedge i < n\}$ *//invariant holds at the beginning of iteration*

```
  i := i + 1;
```

$\{s == \sum_{j=0}^{i-1} j \wedge 0 \leq i-1 \leq n \wedge i-1 < n\}$

$\{s == \sum_{j=0}^{i-1} j \wedge 0 \leq i \leq n\}$ *//invariant is broken*

```
  s := s + i;
```

$\{s == (\sum_{j=0}^{i-1} j) + i \wedge 0 \leq i \leq n\}$ *//invariant is restored*

$\{s == \sum_{j=0}^{i} j \wedge 0 \leq i \leq n\}$ *//invariant is maintained*

```
}
```

$\{i == n \wedge s == \sum_{j=0}^{i} j\}$

$\{s == \sum_{j=0}^{n} j\}$

Hints for finding loop invariants

First: carefully think about the post condition of the loop.

Typically the post-condition talks about a property "accumulated" across a "range" (this is why you are using a loop, right?)

- maximum of all elements in an array
- sort visited elements in a data structure

Hints for finding loop invariants

Second: design a "generalized" version of the post- condition, in which the already visited part of the data is made explicit as a function of the "loop control variable" (generalizing the i.h, remember?)

The loop body may temporarily break the invariant, but must restore it at the end of the body.

Important: make sure that the invariant together with the termination condition really implies your post-condition

Examples of invariants we will need

Max of an array

- All scanned elements of the array are smaller than the max so far

Array searching (unsorted)

- All elements left of the index are different from the value being searched

Array searching (sorted)

- The element is between the lower and higher limits

Sorting

- Everything to the left of the cursor is sorted

List Reversing

- All elements on cursor's left are placed rightly in the result

Weakest Pre-condition Calculus

Hoare Logic – Error prone

Proving the correctness of a program with Hoare Logic

- requires attaching pre-/post-conditions at every instruction
- manual derivations, *i.e.*, lack of automation.

Hoare Logic is not syntax direct: remember the CONSEQUENCE rule.

- There is no algorithm that directly encodes this formal system.

Can we remedy this situation?

- We suspect we can, since we have already used the Why3 tool.

Weakest preconditions

How can we establish the correctness of a program?

One solution: Edsger Dijkstra, 1975

Predicate transformer $\mathrm{WP}(e, Q)$

e expression

Q postcondition

computes the weakest precondition P such that $\{P\}\, e\, \{Q\}$

$\mathrm{WP}(e, Q)$ is recursively defined on the cases of e.

Intuition of WP

$x := 3 * x * y$ $\quad$ $\{\ x$ is even $\}$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$$\texttt{if}\ c\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \quad \{\ Q\ \}$$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$$\texttt{if}\ c\ \texttt{then}\ e_1\ Q \quad \texttt{else}\ e_2\ Q \qquad \{\ Q\ \}$$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$$\texttt{if}\ c\ \texttt{then}\ P_1\, e_1\, Q \quad \texttt{else}\ P_2\, e_2\, Q \qquad \{\ Q\ \}$$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$\{\ \text{if } c \text{ then } P_1 \text{ else } P_2\ \}$ $\quad \texttt{if}\ c\ \texttt{then}\ P_1\, e_1\, Q\ \texttt{else}\ P_2\, e_2\, Q \quad$ $\{\ Q\ \}$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$\{\ \text{if } c \text{ then } P_1 \text{ else } P_2\ \}$ $\quad \texttt{if } c \texttt{ then } P_1\, e_1\, Q \texttt{ else } P_2\, e_2\, Q \quad$ $\{\ Q\ \}$

$\texttt{if } c \texttt{ then } e \quad$ $\{\ Q\ \}$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$\{\ \text{if } c \text{ then } P_1 \text{ else } P_2\ \}$ $\quad \texttt{if } c \texttt{ then } P_1\, e_1\, Q \texttt{ else } P_2\, e_2\, Q \quad$ $\{\ Q\ \}$

$\texttt{if } c \texttt{ then } P\, e\, Q \quad \{\ Q\ \}$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$\{\ \text{if } c \text{ then } P_1 \text{ else } P_2\ \}$ $\quad \texttt{if } c \texttt{ then } P_1\, e_1\, Q \texttt{ else } P_2\, e_2\, Q \quad$ $\{\ Q\ \}$

$\{\ \text{if } c \text{ then } P \text{ else } Q\ \}$ $\quad \texttt{if } c \texttt{ then } P\, e\, Q \quad$ $\{\ Q\ \}$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$\{\ \text{if } c \text{ then } P_1 \text{ else } P_2\ \}$ $\quad \texttt{if}\ c\ \texttt{then}\ P_1\, e_1\, Q\ \texttt{else}\ P_2\, e_2\, Q \quad$ $\{\ Q\ \}$

$\{\ \text{if } c \text{ then } P \text{ else } Q\ \}$ $\quad \texttt{if}\ c\ \texttt{then}\ P\, e\, Q \quad$ $\{\ Q\ \}$

$\texttt{while}\ c\ \texttt{do}\ e \quad$ $\{\ Q\ \}$

Intuition of WP

$\{\ 3xy \text{ is even}\ \}$ $\quad x := 3 * x * y \quad$ $\{\ x \text{ is even}\ \}$

$\{\ Q[s]\ \}$ $\quad x := s \quad$ $\{\ Q[x]\ \}$

$\{\ \text{if } c \text{ then } P_1 \text{ else } P_2\ \}$ $\quad \texttt{if}\ c\ \texttt{then}\ P_1\, e_1\, Q\ \texttt{else}\ P_2\, e_2\, Q \quad$ $\{\ Q\ \}$

$\{\ \text{if } c \text{ then } P \text{ else } Q\ \}$ $\quad \texttt{if}\ c\ \texttt{then}\ P\, e\, Q \quad$ $\{\ Q\ \}$

? $\quad \texttt{while}\ c\ \texttt{do}\ e \quad$ $\{\ Q\ \}$

Definition of WP

$$
\begin{aligned}
\mathrm{WP}(\texttt{skip}, Q) &\equiv Q \\
\mathrm{WP}(x := t, Q) &\equiv Q[x \mapsto t\,] \\
\mathrm{WP}(e_1 \;;\; e_2, Q) &\equiv \mathrm{WP}(e_1, \mathrm{WP}(e_2, Q)) \\
\mathrm{WP}(\texttt{if}\ t\ \texttt{then}\ e_1\ \texttt{else}\ e_2, Q) &\equiv (t \implies \mathrm{WP}(e_1, Q)) \wedge \\
&\qquad (\neg t \implies \mathrm{WP}(e_2, Q))
\end{aligned}
$$

Swimming up the waterfall

```
if odd q then r := r + p;
p := p + p;
q := half q
```

Swimming up the waterfall

```
if odd q then
    r := r + p
else
    skip;
p := p + p;
q := half q
```

Swimming up the waterfall

```
if odd q then
    r := r + p
else
    skip;
p := p + p;
q := half q
```

$Q[p, q, r]$

Swimming up the waterfall

```
if odd q then
    r := r + p
else
    skip;
p := p + p;
```
$Q[p, \texttt{half}\ q, r]$
```
q := half q
```
$Q[p, q, r]$

Swimming up the waterfall

```
if odd q then
    r := r + p
else
    skip;
```

$Q[p + p, \texttt{half}\ q, r]$

$p := p + p;$

$Q[p, \texttt{half}\ q, r]$

$q := \texttt{half}\ q$

$Q[p, q, r]$

Swimming up the waterfall

```
if odd q then
    r := r + p
  Q[p + p, half q, r]
else
    skip;
  Q[p + p, half q, r]
p := p + p;
Q[p, half q, r]
q := half q
Q[p, q, r]
```

Swimming up the waterfall

$$
\begin{array}{l}
\texttt{if odd } q \texttt{ then} \\
\quad Q[p+p, \texttt{half } q, r+p] \\
\quad\quad r := r \texttt{ + } p \\
\quad Q[p+p, \texttt{half } q, r] \\
\texttt{else} \\
\quad Q[p+p, \texttt{half } q, r] \\
\quad\quad \texttt{skip ;} \\
\quad Q[p+p, \texttt{half } q, r] \\
\quad p := p \texttt{ + } p \texttt{ ;} \\
Q[p, \texttt{half } q, r] \\
\quad q := \texttt{half } q \\
Q[p, q, r]
\end{array}
$$

Swimming up the waterfall

$$
\begin{array}{l}
\quad (\texttt{odd}\ q \to Q[p+p, \texttt{half}\ q, r+p]) \wedge \\
(\neg\ \texttt{odd}\ q \to Q[p+p, \texttt{half}\ q, r]) \\
\quad \texttt{if}\ \texttt{odd}\ q\ \texttt{then} \\
\qquad Q[p+p, \texttt{half}\ q, r+p] \\
\qquad\quad r := r\ \texttt{+}\ p \\
\qquad Q[p+p, \texttt{half}\ q, r] \\
\quad \texttt{else} \\
\qquad Q[p+p, \texttt{half}\ q, r] \\
\qquad\quad \texttt{skip}\,; \\
\qquad Q[p+p, \texttt{half}\ q, r] \\
\quad p := p\ \texttt{+}\ p\,; \\
Q[p, \texttt{half}\ q, r] \\
\quad q := \texttt{half}\ q \\
Q[p, q, r]
\end{array}
$$

Weakest Pre-condition Calculus – Example

Consider the following program, lets call it P:

```
if (x > y) then
  z := x;
else
  z := y;
```

1. Compute $\mathbf{WP}(P, z == \mathtt{max}(x, y))$.
2. Is the following triple

$$\{\mathtt{true}\}\,\mathtt{P}\,\{z == \mathtt{max}(x, y)\}$$

   valid?
3. What about

$$\{x \geq 0 \wedge y \geq 0\}\,\mathtt{P}\,\{z == \mathtt{max}(x, y)\}$$

   ? The answer comes in a few minutes.

Definition of WP: loops

$$\begin{array}{l}
\mathrm{WP}(\texttt{while}\ t\ \texttt{do}\ e\ , Q) \equiv \\
\quad \exists J : \texttt{Prop}. \qquad\qquad\qquad\qquad \text{some } \textit{invariant property } J \\
\qquad J \wedge \qquad\qquad\qquad\qquad\qquad\quad \text{that holds at the loop entry} \\
\qquad \forall x_1 \ldots x_k. \qquad\qquad\qquad\quad \text{and is preserved} \\
\qquad\quad (J \wedge \ \ t \to \mathrm{WP}(e, J)) \wedge \qquad\quad \text{after a single iteration,} \\
\qquad\quad (J \wedge \neg t \to Q) \qquad\qquad\quad \text{is strong enough to prove } Q
\end{array}$$

$x_1 \ldots x_k$ references modified in e

We cannot know the values of modified references after n iterations

- therefore, we prove preservation and the post for arbitrary values
- invariant must provide the needed information about the state

Definition of WP: annotated loops

Finding an appropriate invariant is difficult in the general case

- this is equivalent to constructing a proof of Q by induction

We can ease the task of automated tools by providing annotations:

$$\begin{aligned}&\mathrm{WP}(\texttt{while } t \texttt{ invariant } J \texttt{ do } e \texttt{ done}, Q) \equiv \\ &\quad J \wedge \\ &\quad \forall x_1 \ldots x_k. \\ &\qquad (J \wedge \ t \rightarrow \mathrm{WP}(e, J)) \wedge \\ &\qquad (J \wedge \neg t \rightarrow Q)\end{aligned}$$

the given invariant J holds at the loop entry, is preserved after a single iteration, and suffices to prove Q

$x_1 \ldots x_k$ references modified in e

Russian Peasant Multiplication

```
let ref p = a in
let ref q = b in
let ref r = 0 in
while q > 0 invariant J[p,q,r] do
    if odd q then r := r + p;
    p := p + p;
    q := half q
done;
r
```

result = $a * b$

Russian Peasant Multiplication

```
let ref p = a in
let ref q = b in
let ref r = 0 in
while q > 0 invariant J[p,q,r] do
    if odd q then r := r + p;
    p := p + p;
    q := half q
done;
r = a * b
r
```

Russian Peasant Multiplication

```
let ref p = a in
let ref q = b in
let ref r = 0 in
while q > 0 invariant J[p,q,r] do
    if odd q then r := r + p;
    p := p + p;
    q := half q
  J[p, q, r]
done;
r = a * b
r
```

Russian Peasant Multiplication

```
let ref p = a in
let ref q = b in
let ref r = 0 in
while q > 0 invariant J[p,q,r] do
      (odd q → J[p + p, half q, r + p]) ∧
    (¬ odd q → J[p + p, half q, r])
      if odd q then r := r + p;
      p := p + p;
      q := half q
    J[p, q, r]
done;
r = a ∗ b
r
```

Russian Peasant Multiplication

```
  let ref p = a in
  let ref q = b in
  let ref r = 0 in
J[p,q,r] ∧
∀p q r. J[p,q,r] →
  (q > 0 →
      (odd q → J[p + p, half q, r + p]) ∧
    (¬ odd q → J[p + p, half q, r]))) ∧
  (q ⩽ 0 →
    r = a * b)
  while q > 0 invariant J[p,q,r] do
      if odd q then r := r + p;
      p := p + p;
      q := half q
  done;
  r
```

Russian Peasant Multiplication

$$
\begin{array}{l}
J[a,b,0] \wedge \\
\forall p\, q\, r.\ J[p,q,r] \rightarrow \\
\quad (q > 0 \rightarrow \\
\qquad (\texttt{odd}\ q \rightarrow J[p+p, \texttt{half}\ q, r+p]) \wedge \\
\quad\ (\neg\ \texttt{odd}\ q \rightarrow J[p+p, \texttt{half}\ q, r])) \wedge \\
\quad (q \leqslant 0 \rightarrow \\
\qquad r\ =\ a * b) \\
\quad \texttt{let ref}\ p = a\ \texttt{in} \\
\quad \texttt{let ref}\ q = b\ \texttt{in} \\
\quad \texttt{let ref}\ r = 0\ \texttt{in} \\
\quad \texttt{while}\ q > 0\ \texttt{invariant}\ J[p,q,r]\ \texttt{do} \\
\qquad \texttt{if}\ \texttt{odd}\ q\ \texttt{then}\ r := r + p\,; \\
\qquad p := p + p\,; \\
\qquad q := \texttt{half}\ q \\
\quad \texttt{done}\,; \\
\quad r
\end{array}
$$

Theorem

For any e and Q, the triple $\{\mathrm{WP}(e, Q)\}\, e\, \{Q\}$ is valid.

Can be proved by induction on the structure of the program e
w.r.t. some reasonable semantics (axiomatic, operational, etc.)

Corollary

To show that $\{P\}\, e\, \{Q\}$ is valid, it suffices to prove $P \implies \mathrm{WP}(e, Q)$.
(it looks like the Consequence *rule from Hoare Logic, right?)*

This is what Why3 does.

Weakest Pre-condition Calculus – Example, again

Consider the following program, lets call it P:

```
if (x > y) then
  z := x;
else
  z := y;
```

1. Compute $\mathbf{WP}(P, z == \mathtt{max}(x, y))$.
2. Is the following triple

$$\{\mathtt{true}\}\ \mathtt{P}\ \{z == \mathtt{max}(x, y)\}$$

   valid?
3. What about

$$\{x \geq 0 \wedge y \geq 0\}\ \mathtt{P}\ \{z == \mathtt{max}(x, y)\}$$

   ?

Weakest Pre-condition Calculus – Exercise

Consider the following program, lets call it S:

```
x := y;
y := w;
w := x;
```

1. Using weakest pre-condition calculus, prove the following triple

$$\{P(y) \wedge Q(w)\}\, S\, \{P(w) \wedge Q(y)\}$$

   is valid. (here, P and Q are any properties)